

Notes on Programming in Simplex

by

Ivan Gaydardzhiev

ABSTRACT

Simplex is a compiled language for AArch64 Linux. It has one type: a 32-bit `int`. Pointers, strings, and addresses are integers. The compiler reads a source file and spits out an ELF64 executable. It emits machine code, lays out the executable, and sets the entry point. These notes describe the language, the compiler, and the way programs are written when everything is an integer. The compiler will eventually be written in Simplex. It will compile itself, and its output will compile itself again into the same bytes.

1. Introduction

Simplex is a small compiled language for AArch64 Linux. It has five keywords, `int`, `if`, `else`, `while` and `return`, and one type: a 32-bit integer. Pointers, string literals and addresses are integers too. The language is small because it is meant to compile itself: every construct added to Simplex must eventually be implemented again in the Simplex compiler that replaces the C compiler.

The compiler is a recursive-descent parser feeding directly into an AArch64 code emitter. The parser builds an AST, the code generator walks it once, and each node emits A64 instructions into a buffer. The buffer becomes an ELF64 executable: text at `0x400000`, read-only data at `0x500000`, and BSS at `0x600000`. The entry point is recorded after the runtime helpers are emitted.

Forward calls, including recursion, are patched in one final pass. The calling convention is confined to three functions `emit_fn_entry`, `emit_fn_exit`, and `emit_call` each responsible for one part of the ABI. The runtime exposes `__itoa`, `__out`, and `__putstr`; they sit in the function table and are called through `BL` like any other function.

Most constructs did not survive the requirement to pay for themselves twice: once in the C compiler and once in the Simplex compiler that replaces it. What remains is small enough to read in an afternoon.

2. One type

Every value in Simplex is an `int`, and an `int` is 32 bits wide. The compiler lays out the process so that every address fits in 32 bits, with code at `0x400000`, string literals at `0x500000` and global variables and the heap at `0x600000`. An address is therefore an integer like any other, and a variable that holds a pointer needs no type of its own.

```
int x = 8;
int *p = &x;
putint(*p);
putstr("\n");
*p = 16;
putint(x);
putstr("\n");
```

The listing is `ptr.x`. It prints 8 and then 16.

The star in `int *p` is accepted by the parser and recorded in the node, but the code generator never reads it back. A variable declared with a star and one declared without it compile to the same instructions. The star is a note for the reader. I have kept it because a function signature that says `int *buf` tells the next person something that `int buf` does not, and it costs the compiler one `if`.

A string literal is also an integer. It evaluates to the address of its bytes in the read-only segment, so a variable can hold one and `putstr` can print it.

```
int greeting = "Hello from the Simplex World!\n";
putstr(greeting);
```

There are no casts because there is nothing to cast between. Indexing with brackets is defined as a load from the base plus four times the index, which is the only scaling the language knows about.

```
int *p = &buf0;
*p = 10;
p[1] = 20;
```

In this fragment `p[1]` stores into the word that follows `buf0`. This works only because the program declares four adjacent globals, as `arr.x` does, and the compiler assigns them consecutive slots. Reading bytes goes through `bload`, which takes a base and an index and ignores the scale.

```
int msg = "hello\n";
int i = 0;
int c = bload(msg, i);
while (c != 0) {
    putint(c);
    putstr("\n");
    i = i + 1;
    c = bload(msg, i);
}
```

Pointer arithmetic with `+` and `-` is plain integer arithmetic and moves by single bytes. Subtracting two pointers returns a byte distance. That is why `q - p` in the allocator example below prints 16 for two blocks of 16 bytes.

The price of this design is that the compiler cannot catch a pointer used as a number or a number used as a pointer. I accepted that price because a type checker is a large piece of code to write in a language that has no structures to build it from.

3. Control flow and functions

The keywords are `int`, `if`, `else`, `while` and `return`. A condition is an integer, zero is false, and any other value is true. An `else` followed by an `if` needs no special treatment, because the parser sees `else`, looks at the next token, and parses a statement. Every declaration needs an initializer, so a variable never holds a value that the programmer did not write.

```

int fact(int n) {
    if (n == 0) { return 1; }
    return n * fact(n - 1);
}

putint(fact(10));
putstr("\n");

```

A function takes up to eight arguments, which is the number of argument registers that AArch64 provides, and returns one integer. A function that reaches its closing brace returns zero. Recursion needs no declaration. The compiler emits a call to a function by name and fixes the branch target after the whole file has been generated, so a function may call one that appears later in the file. A global variable gets no such treatment. It must be declared above every function that uses it, because the compiler reads the file once from top to bottom.

The logical operators deserve a warning. In `a && b` the compiler evaluates both operands and combines them with a bitwise AND, and `||` does the same with OR. Comparison operators produce exactly zero or one, so `x > 0 && y > 0` does what a C programmer expects. A bare `p && q` with `p` equal to 4 and `q` equal to 2 evaluates to zero. I write conditions as comparisons and have not needed anything else. Division and remainder are unsigned, while the ordering comparisons are signed. Programs that divide negative numbers will get unsigned results.

3.1. Locals go at the top

All local variables of a function must be declared before the first `if` or `while`. The rule exists because of how the compiler allocates stack space. A declaration reserves its 16 byte slot at run time, by pushing the initial value, and the compiler records the slot's offset from the frame pointer at compile time. If the declaration sits inside a branch that does not execute, the push never happens and every later local has an offset that points at the wrong place. A declaration inside a loop would push again on every iteration. Straight-line declarations at the top make the compile time count and the run time count agree, and a function written this way reads well anyway. Everything a function owns is listed before it does anything.

```

int putstrn(int s, int n) {
    int i = 0;
    int c = 0;
    int buf = malloc(2);
    int *p = buf;
    p[0] = 0;
    while (i < n) {
        c = bload(s, i);
        p[0] = c;
        putstr(buf);
        i = i + 1;
    }
    return 0;
}

```

This function from the parser prints `n` bytes of a string that has no terminating zero. It builds a two byte string in the heap, patches the first byte in place and prints it. There is no `write` call that takes a length, so this is how a counted string reaches the screen. The

function is not pleasant, and I would rather see that in the source than hide it behind a library routine.

4. Memory

There is no `malloc` and there will not be one. The language provides `balloc`, which takes a byte count, rounds it up to a multiple of four, and returns the old value of a bump pointer after adding the count to it. The pointer lives in the first word of a 64 KB arena in the zero-initialized segment. Nothing is ever freed. The memory comes back when the process exits.

```
int p = balloc(16);
int q = balloc(16);
p[0] = 111;
p[1] = 222;
q[0] = 333;
putint(q - p);
```

An arena with no free is the right allocator for a compiler. The compiler reads a file, builds a tree, walks the tree once and exits, so no node in the tree outlives the run and nothing is gained by tracking when it dies. Each pass allocates in the order it needs, and the whole heap is released in a single system call. The allocator does not check for exhaustion. A program that asks for more than 64 KB will write over whatever follows the arena, so the arena size is part of the contract and has to be raised before a larger input is fed to the parser.

4.1. Structures without structures

The language has no structures, so a record is a block of words and each field is an offset into it. The smallest case in the repository is a tree node of four words that holds a type, a number and two children.

```
int nd_type(int nd) { return nd[0]; }
int nd_num(int nd)  { return nd[1]; }
int nd_ca(int nd)   { return nd[2]; }
int nd_cb(int nd)   { return nd[3]; }

int mknum(int val) {
    int nd = balloc(16);
    nd_set_type(nd, ND_NUM);
    nd_set_num(nd, val);
    nd_set_ca(nd, 0);
    nd_set_cb(nd, 0);
    return nd;
}

int eval(int nd) {
    if (nd_type(nd) == ND_NUM) { return nd_num(nd); }
    return eval(nd_ca(nd)) + eval(nd_cb(nd));
}
```

Building the sum of 10 and 5 and then adding 3 to that tree prints 15 and then 18. The field names exist only as accessor functions, and the offsets exist only inside them. When a layout changes, one function changes. The cost is a call for every field access, which on a compiler that does no inlining is real but small.

The parser in `stage1` uses the same technique at a larger size. A token is five words, holding a type, a number, a pointer to the text, a length and a line. A syntax node is a fixed block of 232 bytes, which is 58 words. The first nine words hold the node type, a numeric value, a string pointer and length, a pointer flag, three child slots and a count. The next sixteen words hold a list of children. After that come a parameter count and two sixteen word arrays, one for parameter names and one for their pointer flags.

```
int nd_nch(int n) { int *p = n; return p[8]; }
int nd_ch(int n, int i) { int *p = n; return p[9 + i]; }
int nd_npar(int n) { int *p = n; return p[25]; }
int nd_par_str(int n, int i) { int *p = n; return p[26 + i]; }
```

Every node is the same size whether it is a number or a function, so a node needs no size field and the allocator needs no size classes. Most of the 232 bytes of a number node go unused. I paid that to avoid writing a second allocation path. The arrays are capped at sixteen entries and the code that fills them does not check the cap, so a block with seventeen statements damages the node that holds it. The limits hold for the programs compiled today and will be lifted before the compiler compiles itself.

5. Built-ins instead of a library

Six names are special. `putint` prints an integer in decimal, `putstr` prints a zero-terminated string, `getc` reads one byte from standard input, `bload` and `bstore` read and write a single byte, and `balloc` allocates. The code generator recognizes them by name, before it looks in its table of user functions, and emits their instructions inline or as a branch to one of three runtime routines. A program cannot redefine them.

The underlying interface is the Linux system call convention. The call number goes in `X8`, arguments go in `X0` to `X5`, and `SVC #0` enters the kernel. Simplex uses three calls, `write` as 64, `read` as 63 and `exit` as 93. `putstr` finds the length of its argument by scanning for the zero byte and then issues one `write`. `putint` converts the value into a scratch buffer at the top of the data segment, filling it from the right, and issues one `write`. Neither buffers anything, so a program that prints a thousand numbers makes a thousand system calls. On a language whose main job is to compile itself this is fine, and it keeps the three routines short enough to read.

I considered writing these routines in Simplex and linking them in as a prelude. The routines are the only code in the system that is shared by every program, and putting them in the compiler means every output file is complete as it stands. With a prelude, I would need either a linker or a rule for where the prelude text gets prepended, and either one is more machinery than three short functions written in machine code.

6. The compiler

The whole pipeline is the body of `main`. It reads the file, builds a lexer, parses, generates code and writes the output.

```

src = rc(in);
lx = lx_new(src);
prog = parse(lx);
cg_init(&cg);
codegen(prog, &cg);
emit_elf(&cg, out);

```

The parser is recursive descent with one function for each level of precedence. From loosest to tightest they are `por`, `pand`, `peq`, `pcmp`, `padd`, `pmul`, `punary` and `pprim`, with assignment handled one level above them in `pexpr`. Each binary level has the same shape. It parses the next tighter level, then loops while it sees one of its own operators and wraps what it has in a new node. Postfix brackets are handled in `punary`, which is why `*p[1]` means the dereference of `p[1]`. The parser builds a tree of heap nodes and the code generator walks that tree once. No pass modifies it.

The code generator keeps no register allocator. Every expression leaves its result in `X0`. For a binary operation the left side is evaluated and pushed, the right side is evaluated, the left side is popped into `X1`, and one instruction combines them.

```

case ND_BIN: {
    Val va = gen_expr(g, n->a, env);
    val_rval(g, va);
    a64_push_w0(g);
    Val vb = gen_expr(g, n->b, env);
    val_rval(g, vb);
    a64_pop_wt(g, 1);
    switch (n->op) {
    case TK_PLUS:
        a64_add_rr(g, 0, 1, 0);
        break;

```

The one refinement is that `gen_expr` returns a small value that describes where the result is rather than emitting a load at once. A value may be a constant, a slot in the frame, an absolute address in the data segment, or already in `X0`. A constant that is going to be stored costs one instruction and a variable that is going to be assigned costs none for the load. The function `val_rval` forces a value into `X0` when the caller needs it there. I find this the most useful idea in the compiler. It removes the redundant loads that make naive output slow, and it costs about forty lines.

6.1. Calling convention in three functions

All the logic that touches the calling convention is in `emit_fn_entry`, `emit_fn_exit` and `emit_call`, and nowhere else. The entry saves `X29` and `X30`, sets the frame pointer, reserves a 16 byte slot for each parameter and spills each argument register into its slot. The exit sets the stack pointer back to the frame pointer before it restores the saved pair and returns. That single instruction makes the epilogue correct no matter how many locals were pushed or on which path.

The call evaluates each argument in turn, pushes it, loads the pushed values into `X0` through `X7` and pops the stack. Every push takes a full 16 byte slot, because AArch64 requires the stack pointer to be 16 byte aligned whenever it is used for an access, and a call that arrives with a misaligned stack faults. Spending twelve bytes of padding on every word is wasteful,

and I accept it because one rule that holds everywhere is easier to check than a rule that holds when the count is even.

A call is emitted as **BL** with a zero offset, and the compiler writes the call site and the callee's name in a list. After the last statement has been generated, one loop walks the list, looks up each name and patches the offset. A name that is not found is a compile error. Recursion and calls to later functions work for the same reason.

6.2. The output file

The ELF file has a 64 byte header, three program headers of 56 bytes each, and no section headers, so the headers occupy 232 bytes. The first program header maps the file from offset zero to **0x400000** as readable and executable, which puts the headers inside the text segment and makes the first instruction land at **0x4000e8**. The second maps the string literals at **0x500000** as read-only, after padding the file to a page boundary. The third describes the data segment at **0x600000**, which has no bytes in the file and is zeroed by the kernel. It holds 1024 four byte globals, 32 bytes of scratch space, the bump pointer and the 64 KB arena.

The three runtime routines come first in the text, so the entry point is the address after them. In every file I have compiled it is **0x4001d0**, because the routines occupy 232 bytes. A program that prints one string is 4127 bytes long. Almost all of that is the padding that puts the 31 bytes of the string at the next page.

The compiler does no optimization. The same source always yields the same bytes, with no dependence on a hash order, a timestamp or the state of the machine, so comparing two outputs with **cmp** is a complete test. That is the test the bootstrap needs.

7. No linker

The top level of a Simplex file is a sequence of statements that run in order when the program starts. A function definition at the top level compiles to a jump over its body, so defining a function executes nothing, and every other statement executes where it stands. A global declaration stores its initial value into the data segment at that point in the run.

It follows that a program can be assembled from several files by concatenating them in dependency order. The lexer, the parser and the code generator of a compiler can live in separate files and be joined with **cat**, and the result is a legitimate input. No symbol table crosses a file boundary, so there are no relocations to resolve and no object format to define. The **cat** command is the linker, and since it ships with every system there is nothing to bootstrap.

The cost is that there is no separate compilation and the compiler reads the whole program on every run. The compiler holds a global table of 1024 variables, 256 functions and at most 4096 call sites, and a program that exceeds any of these is rejected. For a compiler whose own source will be a few thousand lines I consider that a reasonable ceiling.

8. Self hosting

The compiler is complete when it compiles its own source and produces a binary that does the same again, with the second generation identical to the first. Stage one of that work is two files. The first, **lex.x**, is the lexer. It has a single function **lx_one** that consumes one token from a buffer, recognizes keywords by comparing the text to zero-terminated keyword strings, and returns a 20 byte token from the arena. The second, **parse.x**, holds the lexer

again and a recursive descent parser that follows the C version function for function and builds the 232 byte nodes described above.

A test driver in `parse.x` feeds the parser a string containing the factorial program and prints the tree.

```
!!! AST !!!
PROG
  FN fact
    BLOCK
      IF
        BIN
          ID n
          NUM 0
        BLOCK
          RETURN
          NUM 1
      RETURN
      BIN
        ID n
        CALL fact
        BIN
          ID n
          NUM 1
    CALL putint
    CALL fact
    NUM 10
  CALL putstr
  STR
```

A parser written in Simplex, compiled by the C compiler, running on AArch64, reads Simplex source and produces the tree that the C parser would produce. I read that output as evidence that the language is large enough for the job, since a lexer and a parser already need recursion, string comparison, a heap and records built from words.

Two pieces remain. The first is `codegen.x`, which is the emitter and the ELF writer. It is the largest part, because the C version relies on a growable buffer, a table of functions and a table of globals, and each of those has to be rebuilt from words and offsets. The second is the substitute for structures. Writing the accessor functions by hand worked for a token and a node, but the code generator has more record types and I expect to generate those accessors rather than type them. When both exist, the three files are joined with `cat`, with their test drivers removed and one entry point added at the end. The C compiler compiles the joined source into a first stage, and the Simplex compiler that results compiles the same source into a second stage. If the two files are identical, the compiler has reproduced itself.

Availability. The compiler, the examples and the stage one sources are licensed under the GPL version 3 and are available in the project repository at <https://github.com/gaidardzhiev/simplex>